Applied Soft Computing Journal 171 (2025) 112783

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Robust graph contrastive learning with multi-hop views for node classification

Yutong Wang [a], Junheng Zhang [a], Ren Cao [a], Minhao Zou [a], Chun Guan [a], Siyang Leng [a,b,*]

[a] Institute of AI and Robotics, Academy for Engineering and Technology, Fudan University, Shanghai 200433, China
[b] Research Institute of Intelligent Complex Systems, Fudan University, Shanghai 200433, China

ARTICLE INFO

Keywords:
Self-supervised learning
Graph contrastive learning
Multi-hop
Node classification

ABSTRACT

Contrastive learning has been proven successful in graph self-supervised learning by addressing label scarcity in real-world applications. Most existing methods fail to effectively incorporate multi-hop information into the augmented views but only focus on augmenting the raw input data, which incurs massive computational and memory costs. To address such limitations, we propose a novel approach, Multi-hop Views Graph Contrastive Learning (MHVGCL), that enhances the node classification performance for graphs. Specifically, in contrast to existing methods that generate multiple augmented heads via neural networks, our approach generates these heads by exploiting multi-hop information, which is obtained iteratively from a single output head. This technique can extract more comprehensive structural information without destroying the graph topology. We further devise a multi-hop contrastive loss function to maximize agreements among the multi-hop views of the same node, while minimizing them among different nodes. This design contributes to more robust representation learning that keeps structural attributes invariant under different augmented views. Numerical experimental results on a variety of benchmarks demonstrate the significant superiority of our approach over other advanced methods, by learning more discriminative node representations even with extremely limited labels.

1. Introduction

Neural networks renowned for their effectiveness in uncovering complex relationships within data have been widely explored and applied across numerous fields, such as computer vision [1,2], natural language processing [3], materials science [4–7], and etc. Building upon these advancements, adapting neural networks to graph-structured data has emerged as a significant research direction. In recent years, graph neural networks (GNNs) have attracted considerable attention owing to their excellent performance across diverse graph mining tasks, including node classification [8–10], link prediction [11], and graph classification [12]. Kipf and Welling [8] introduced the Graph Convolutional Network (GCN), which leverages spectral graph theory to perform semi-supervised learning on graph-structured data, achieving state-of-the-art results on node classification benchmarks. Later, Velickovic et al. [9] proposed the Graph Attention Network (GAT), which employs attention mechanisms to assign different weights to neighboring nodes, further improving node classification performance. Recently, Zou et al. [10] demonstrated that incorporating similarity metrics enhances node embeddings in heterophilic graph contexts. While these approaches achieve impressive accuracy, most existing GNNs are trained in a (semi-)supervised manner, heavily relying on massive labeled data, which produces substantial computational costs and human annotation efforts. Particularly, in real-world applications, this inevitably leads to significant expenditures of time and resources in collecting such labeled data.

Graph contrastive learning (GCL) has emerged as an effective graph self-supervised learning paradigm to address the label scarcity problem. For instance, You et al. [13] proposed the Graph Contrastive Learning (GraphCL) framework, which generates augmented views through random topology disturbances such as node dropping and edge perturbation. Zhu et al. [14] introduced the Deep Graph Infomax (DGI), which uses feature masking to maximize mutual information between graph-level and node-level representations. By utilizing data augmentation and contrastive objectives, GCL allows high-quality node representations to be learned in a self-supervised manner with limited label information. Therefore, generating augmented views that sufficiently incorporate node features and graph structure information is crucial for obtaining superior performance on downstream tasks such as node classification. Intuitively, augmented views containing

* Corresponding author at: Institute of AI and Robotics, Academy for Engineering and Technology, Fudan University, Shanghai 200433, China.
E-mail addresses: mhzou22@m.fudan.edu.cn (M. Zou), chunguan@fudan.edu.cn (C. Guan), syleng@fudan.edu.cn (S. Leng).

https://doi.org/10.1016/j.asoc.2025.112783
Received 8 August 2024; Received in revised form 24 December 2024; Accepted 16 January 2025
Available online 23 January 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

node attributes and structure details enable more effective contrast between positives and negatives for self-supervised graph representation learning.

Most GCL works generate different views through two typical techniques, i.e., topology augmentation and feature augmentation. Specifically, as graph topology contains essential structural information, random topology disturbances generating topology augmentations, such as node dropping [13], edge removing [15] and adding [13], fail to preserve the original graph topology and consequently degrade the quality of learned representations on certain downstream tasks. On the other hand, pure feature augmentations like masking [14] and dropout [13] only depend on the node features but neglect structural information, leading to suboptimal performance. Moreover, these methods fail to consider multi-hop information, which preserve the graph topology and can bring in both local and global topology and feature information. Therefore, how to effectively exploit multi-hop information to strengthen the robustness and performance of GCL remains an open question. Furthermore, these augmentation methods, originated from the computer vision field and only augmented the raw input data, entail substantial computational and memory costs.

To overcome the constraints, we present an innovative framework termed Multi-hop Views Graph Contrastive Learning (MHVGCL). In contrast to conventional multi-head approaches that create multiple augmented views with unshared learnable parameters through a few neural networks, multi-hop information can be incorporated into GCL from a single neural network output head. This technique mitigates issues of excessive computational cost and memory overhead, compared to performing augmentations on raw data, as the dimensionality of the output head is always much lower. Concretely, it generates multiple augmented heads by exploiting multi-hop information, which are obtained iteratively from the single output head. This technique can extract more comprehensive structural information without destructing the graph topology. We further devise a multi-hop contrastive loss to maximize agreements among the multi-hop views of a same node, while minimizing them among different nodes. This design contributes to more robust representation learning that keeping structural attributes invariant under different augmented views. In addition, we design an alternative version MHVGCL-2V of MHVGCL which contains only the first augmented view and the last augmented view during training and downstream task, which further save memory and time cost compared with the multi-view version MHVGCL-MV.

The key contributions of this work can be summarized as follows:

- We propose a graph contrastive learning framework, called MHVGCL, that generates augmented views directly from a single output head, instead of exploiting multi-head outputs produced from the raw data. MHVGCL includes two versions: MHVGCL-MV, which leverages all augmented views, and MHVGCL-2V, which contains only the first and the last augmented view, significantly reducing computational and memory overhead.
- We design an augmentation method for graph data that generates multiple augmented views by integrating multi-hop information. This technique can extract more comprehensive structural information without destructing the graph topology.
- We introduce a multi-hop contrastive loss function to maximize agreements among the multi-hop views of a same node, while minimizing them among different nodes. This design contributes to more robust representation learning that keeping structural attributes invariant under different augmented views.

Experimental evaluations on various benchmarks validate the advancement of our approach over other advanced methods, by learning more discriminative node representations even with extremely limited labels.

The rest of the paper is organized as follows. In Section 2, we discuss related works, focusing on two key areas: Graph Neural Networks Incorporating Multi-hop Information and Graph Contrastive Learning. Section 3 presents the Methodology, detailing the Preliminaries, Base View Generation, Multi-hop Views, and the Multi-hop Contrastive Loss. In Section 4, we outline the Experiments, including the Datasets, Baselines, and Experimental Settings. The results are presented in Section 5, covering Node Classification Performance, Computational and Memory Efficiency Comparison (with sub-comparisons to NCLA and other self-supervised methods), Ablation Study, Hyperparameter Analysis, Visualization of Node Embeddings, and Discussion. Finally, we conclude the paper in Section 6, summarizing our findings and suggesting directions for future research.

2. Related works

Regarding recent developments in GNNs and GCLs, we concisely review existing works mainly of two areas: (1) recent graph neural networks that incorporate multi-hop information; (2) recent graph contrastive learning approaches.

2.1. Graph neural networks incorporating multi-hop information

In graph neural networks, the receptive field of a single layer is restricted to one-hop connections. This restriction can be released by stacking multiple layers to exploit multi-hop information, which enlarges the receptive field. Existing works have incorporated multi-hop context in different ways. MHGNNs [16] address the issue of narrow receptive field for graph-structured data by concatenating hop-level features and reusing neighbors information to capture further hop-level features effectively. MAGNA [17] merges multi-hop context information into each layer of GNN attention computation, which learns more informative attention distribution and captures large-scale structural information in every layer. AGDNs [18] integrate smaller multi-hop node representations with learnable and generalized weighting coefficients, rather than combining large and dense powers of the transition matrix with predefined weighting coefficients. UniG-Encoder II [19] performs a multi-hop aggregation process to iteratively update the node embeddings, and generates final embeddings by concatenating them via a 1×1 convolution kernel. In the proposed method, we perform multi-hop aggregation on a single output head to generate multi-hop views for graph contrastive learning, which is different from the above GNN paradigms.

2.2. Graph contrastive learning

Graph contrastive learning is currently a trending topic in graph representation learning, which draws inspiration from prominent contrastive learning frameworks originated in computer vision, by utilizing data augmentation and contrastive objectives, such as CPC [1], SimSiam [20], and MoCo [2]. Existing GCL works have incorporated a variety of augmentation and contrasting strategies [21]. NCLA [22] introduces a novel neighbor contrastive loss for node-node GCL to allow for multiple positives per anchor, employing network topology as the supervised signals. DGI [23] employs the row-wise shuffling of node attributes to contrast node representations between the original and corrupted graph. GMI [24] contrasts the input neighborhood feature and hidden representation of each node. GRACE [15] utilizes a contrastive objective at the node level after generating two graph views through the removal of edges and the masking of attributes. Unlike GRACE's uniform perturbation, GCA [14] performs adaptive augmentation according to the edges' and features' importance scores. TriCL [25] constructs a tri-directional contrast, combining node-, group-, and membership-level contrasts. MVGRL [26] uses graph diffusion to generate an additional structural view of a sample graph which along with a regular view are sub-sampled and fed to two dedicated GNNs followed by a shared MLP to learn node representations. In this work, we demonstrate that incorporating the multi-hop views can further enhance the node classification performance.

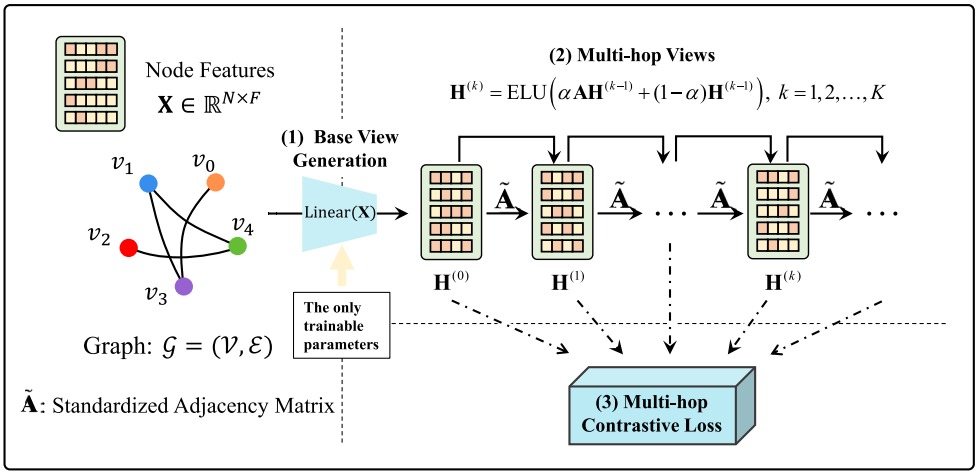


Fig. 1. The proposed multi-hop views graph contrastive learning (MHVGCL) framework.

3. Methodology

The overall architecture of the proposed Multi-hop Views Graph Contrastive Learning framework is depicted in Fig. 1. It consists of three key components: (1) base view generation, (2) multi-hop views, (3) multi-hop contrastive loss. We first utilize linear transformation to generate the base view containing relatively low-dimensional node representations. This base view is then used to generate multiple augmented views by integrating multi-hop information. A multi-hop contrastive loss is subsequently used to maximize agreement between embeddings of positive pairs while minimizing agreement between negative pairs across all the multi-hop views. The learned node representations can then be further utilized for downstream node classification and clustering tasks. The technical details and design of each component are described in the following subsections.

3.1. Preliminaries

Notations. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected graph, consisting of a node set $\mathcal{V} = \{v_1, \ldots, v_N\}$ with N elements and an edge set $\mathcal{E} = \{e_1, \ldots, e_M\}$ with M elements, where each element e_i in the edge set $\mathcal{E}$ represents a connection between two nodes in the node set $\mathcal{V}$.

We define the raw node feature matrix $\mathbf{X} \in \mathbb{R}^{N\times F}$, whose ith F-dimensional row-component vector $\mathbf{x}_i$ corresponds to node v_i's features. Denote the cardinality of set $\mathcal{S}$ as $|\mathcal{S}|$. The incidence matrix $\mathbf{B} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{E}|}$ can concisely represent the graph $\mathcal{G}$, with $\mathbf{B}_{ij} = \begin{cases} 1, & \text{if } v_i \in e_j \\ 0, & \text{otherwise} \end{cases}$, where $e_1, \ldots, e_{|\mathcal{E}|}$ is an ordered sequence of all edges. The degrees are defined as $d(v_i) = \sum_j \mathbf{B}_{ij}$ and $\delta(e_j) = \sum_i \mathbf{B}_{ij}$ for nodes and edges respectively. Diagonal matrices of these degrees are written as $\mathbf{D}_{\mathcal{V}} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ and $\mathbf{D}_{\mathcal{E}} \in \mathbb{R}^{|\mathcal{E}|\times|\mathcal{E}|}$. As introduced in [19], the standardized adjacency matrix incorporating self-loops is $\widetilde{\mathbf{A}} = \left[\mathbf{I} + \mathbf{D}_{\mathcal{V}}\right]^{-1}\left[\mathbf{I} + \mathbf{B}\mathbf{D}_{\mathcal{E}}^{-1}\mathbf{B}^{\top}\right]$. We focus on semi-supervised node classification task, where only a subset of nodes $\mathcal{V}_L$ are assigned with labels $\mathcal{Y}_L$. These labeled data are referred to as $\mathcal{D}_L = (\mathcal{V}_L, \mathcal{Y}_L)$, while the remaining unlabeled nodes are denoted by $\mathcal{D}_U = (\mathcal{V}_U, \mathcal{Y}_U)$, where $\mathcal{Y}_U$ corresponds to their unknown true labels.

To facilitate reference, we further summarize all the symbols used in the paper as follows (see Table 1).

Problem Formulation. Our objective is to learn useful node embeddings using a linear transformation combined with an augmentation function $\mathbf{g}$. This function takes the node features $\mathbf{X}$ and graph structure $\tilde{\mathbf{A}}$ as input, and outputs a multi-hop views representation $\mathbf{H} = \mathbf{g}(\mathrm{Linear}(\mathbf{X}), \tilde{\mathbf{A}}) \in \mathbb{R}^{N\times(F'\cdot K)}$ in a low-dimensional space. Here the linear transformation $\mathrm{Linear}(\mathbf{X})$ reduces the dimensionality of features from F to F' ($F' \ll F$) and K is the number of augmented views.

Table 1
Summary of symbols and their meanings.

Symbols	Notations
$\mathcal{G}$	Undirected graph
$\mathcal{V}$	Node set
$\mathcal{E}$	Edge set
$\mathbf{X}$	Raw node feature matrix
$\lvert\mathcal{S}\rvert$	The cardinality of set $\mathcal{S}$
$\mathbf{B}$	Incidence matrix
$d(v_i)$	Degree of node v_i
$\delta(e_j)$	Degree of edge e_j
$\widetilde{\mathbf{A}}$	Standardized adjacency matrix incorporating self-loops
$\mathcal{D}_L$	Labeled data
$\mathcal{D}_U$	Unlabeled data
$\mathbf{g}$	Augmentation function
$\mathbf{H}$	Multi-hop views representation
K	The number of augmented views
F	The original dimensionality of features
F'	The reduced dimensionality of features
ψ_θ	An L_2-regularized logistic regression classifier
$\mathbf{h}_i$	The ith row of $\mathbf{H}$
y_i	Known label for the nodes in $\mathcal{V}_L$
$\tilde{\ell}$	A function measuring the distance between predicted and true labels.
$\mathbf{H}^{(0)}$	The base view
$\mathbf{S}$	The bias term
$\mathbf{W}$	A learnable weight matrix
$\mathbf{H}^{(k)}$	Multi-hop views

These generated node embeddings can be adapted to fulfill various downstream tasks including node classification considered in this work.

Let ψ_θ signify an L_2-regularized logistic regression classifier parameterized by θ. The objective for semi-supervised node classification is to minimize the loss $\mathcal{L}_{\mathrm{cls}}$

$$\min_{\theta} \mathcal{L}_{\mathrm{cls}}(\theta, \mathbf{H}, \mathcal{D}_L) = \min_{\theta} \sum_{(v_i, y_i)\in\mathcal{D}_L} \tilde{\ell}\left(\psi_\theta(\mathbf{h}_i), y_i\right), \tag{1}$$

where $\mathbf{h}_i$ is the ith row of $\mathbf{H}$ denoting the learned embedding of node v_i, and y_i represents the known label for the nodes in $\mathcal{V}_L$. $\tilde{\ell}$ is a function measuring the distance between predicted and true labels.

3.2. Base view generation

In MHVGCL, all augmented views are generated from the same base view, which is obtained through a linear transformation. The parameters of this linear transformation are learnable. In fact, these parameters are the only ones that need training throughout the whole procedure, which significantly mitigates computational and memory burdens comparing to the methods generating multiple augmented views separately

but from raw input data. The output of the linear transformation, serving as the basis, is subsequently fused with multi-hop information iteratively, without generating more learnable parameters.

The base view, $\mathbf{H}^{(0)}$, can thus be expressed as

$$\mathbf{H}^{(0)} = \mathbf{X}\mathbf{W} + \mathbf{S}, \tag{2}$$

where $\mathbf{S} \in \mathbb{R}^{N\times F'}$ is the bias term and $\mathbf{W} \in \mathbb{R}^{F\times F'}$ denotes a learnable weight matrix projecting input features into the embedding space of base view. The bias term, practically implemented as a broadcast of a bias vector $\mathbf{s} \in \mathbb{R}^{1\times F'}$, offers flexibility in shifting the base view embeddings as needed.

3.3. Multi-hop views

Most GNNs incorporate multi-hop information naturally by propagating node features across multiple layers. This mechanism allows for aggregating the information from farther nodes in high order neighborhoods. More informative node representations encoding both local feature and global structure information, can be learned through stacking more layers, which on the other hand may cause problems such as over-smoothing. Different from the conventional setting, we explicitly use the kth power ($k = 1, 2, \dots$) of the adjacency matrix, which encodes the numbers of k-hop paths connecting pairs of nodes, to aggregate features from multi-hop neighbors, and eliminate the need for additional layers and extra trainable parameters. By exploiting each hop, useful information can be propagated and encoded into the final representation. We further fuse together information from all hops through an aggregation mechanism to create the final multi-hop views.

To be concrete, the multi-hop views $\mathbf{H}^{(k)}$ are iteratively generated according to the following equation:

$$\mathbf{H}^{(k)} = \mathrm{ELU}\left(\alpha\tilde{\mathbf{A}}\mathbf{H}^{(k-1)} + (1-\alpha)\mathbf{H}^{(k-1)}\right), \tag{3}$$

where $k = 1, 2, \dots, K$ and $\mathbf{H}^{(k)} \in \mathbb{R}^{N\times F'}$. Notice that all the K multi-hop views are sequentially generated based on $\mathbf{H}^{(0)}$, with simple iterations. The trade-off parameter α controls the fusion proportion between the two consecutive augmented views. ELU refers to the Exponential Linear Unit activation function.

Finally, for MHVGCL-MV, the embeddings of all views are concatenated to generate the output node representation $\mathbf{H}$, i.e.,

$$\mathbf{H} = [\mathbf{H}^{(0)}, \mathbf{H}^{(1)}, \dots, \mathbf{H}^{(K)}]. \tag{4}$$

While for MHVGCL-2V, only the embeddings of the first and the last views are concatenated, i.e.,

$$\mathbf{H} = [\mathbf{H}^{(0)}, \mathbf{H}^{(K)}]. \tag{5}$$

The obtained $\mathbf{H}$ is then used to calculate the contrastive loss, which is employed to update the parameters $\mathbf{W}$ and $\mathbf{S}$ via gradient descent. The learned node embeddings $\mathbf{H}$ are subsequently conveyed to downstream tasks.

This component in MHVGCL serves as the key advancement due to its efficiency in generating the augmented views, compared to traditional graph augmentation techniques. It can produce meaningful augmented views that integrate the rich multi-hop graph information but avoid bringing in excessive computational costs. This technique of iterating multi-hop views based on a single neural network output further preserves the original graph topology, without harming the quality of structural information as in the perturbation-based augmentation methods.

3.4. Multi-hop contrastive loss

We devise a multi-hop contrastive loss based on the work in InfoNCE [2], where a commonly used loss in GCL is proposed. InfoNCE realizes contrasting learning by enhancing discriminability through pulling positive pairs close while pushing negative pairs apart. Specifically, the embeddings of a node v_i across two different views, $\mathbf{h}_i^{(k)}$ and $\mathbf{h}_i^{(l)}$ ($k \neq l$), form a positive pair. And the embeddings of different nodes in either two same or two different views are treated as negative pairs.

The similarity between a node pair (v_i, v_j) across views 1 and 2 is defined as

$$s_{ij}^{(1,2)} = \mathrm{sim}\left(\mathbf{h}_i^{(1)}, \mathbf{h}_j^{(2)}\right)/\tau, \tag{6}$$

where $\mathrm{sim}\left(\mathbf{z}_i, \mathbf{z}_j\right) = \left(\mathbf{z}_i\mathbf{z}_j^\top\right)/\left(\|\mathbf{z}_i\|\,\|\mathbf{z}_j\|\right)$, and τ is a temperature parameter.

The InfoNCE loss for node v_i contrasting positive pairs against negative pairs is written as,

$$\ell\left(\mathbf{h}_i^{(1)}, \mathbf{h}_i^{(2)}\right) = -\log\frac{\exp\left(s_{ii}^{(1,2)}\right)}{\underbrace{\exp\left(s_{ii}^{(1,2)}\right)}_{\text{positive pair}} + \mathrm{Neg}}, \tag{7}$$

where the negative component Neg sums both intra- and inter-view negatives:

$$\mathrm{Neg} = \sum_{j=1}^{N} 1_{[j\neq i]}\Big[\underbrace{\exp\left(s_{ij}^{(1,1)}\right)}_{\text{intra-view negative pair}} + \underbrace{\exp\left(s_{ij}^{(1,2)}\right)}_{\text{inter-view negative pair}}\Big]. \tag{8}$$

$1_{[j\neq i]} \in \{0, 1\}$ is an indication function that equals to 1 iff $j \neq i$.

Considering the symmetry of views, the total contrastive loss $\mathcal{L}_{\mathrm{ssl}}$ between views k and l, averaged over all nodes, is

$$\mathcal{L}_{\mathrm{ssl}} = \frac{1}{2N}\sum_{i=1}^{N}\left[\ell\left(\mathbf{h}_i^{(k)}, \mathbf{h}_i^{(l)}\right) + \ell\left(\mathbf{h}_i^{(l)}, \mathbf{h}_i^{(k)}\right)\right]. \tag{9}$$

In our proposed MHVGCL-MV where more than two views are available, we randomly select one view $\mathbf{H}^{(l)}$ as the pivot. The total multi-hop contrastive loss averages all the pairwise losses between the pivot and every other view:

$$\mathcal{L} = \frac{1}{K-1}\sum_{k=0,k\neq l}^{K}\mathcal{L}_{\mathrm{ssl}}(\mathbf{H}^{(k)}, \mathbf{H}^{(l)}). \tag{10}$$

Notably, both the augmented views and node embeddings are learned in an end-to-end manner in MHVGCL, providing flexibility and ease of use. The detailed algorithmic description of MHVGCL is presented in **Algorithm 1**.

Algorithm 1 : MHVGCL

Require: standardized adjacency matrix $\tilde{\mathbf{A}}$, feature matrix $\mathbf{X}$, number of augmented views K, number of training epochs $\mathcal{T}$

```
for t = 1, 2, ···, T do
    Generate the base view H^(0) with Eq. (2);
    for k = 1, 2, ···, K do
        Generate the k-th augmented view H^(k) iteratively with Eq. (3);
    end for
    Concatenate the augmented views to generate the final embeddings with Eq. (4) for MHVGCL-MV or Eq. (5) for MHVGCL-2V;
    Compute multi-hop contrastive loss with Eq. (10);
    Update parameters by applying gradient descent to minimize L;
end for
Generate embeddings with optimal parameters with Eqs. (2), (3), and (4) or (5).
```

Ensure: Embeddings $\mathbf{H}$

Table 2
Statistics of five graph datasets.

Datasets	CiteSeer	Cora	PubMed	Coauthor-CS	Amazon-Photo
$\lvert\mathcal{V}\rvert$	3,327	2,708	19,717	18,333	7,650
$\lvert\mathcal{E}\rvert$	9,228	10,556	88,651	163,788	238,162
# Features	3,703	1,433	500	6,805	745
# Classes	6	7	3	15	8

Table 3
Hyperparameter settings of MHVGCL-2V.

Datasets	# Augmented views	# Embedding dimension	Temperature	Learning rate	Weight decay
Cora	7	128	5.0	1e−2	5e−5
CiteSeer	6	128	5.0	1e−3	1e−4
PubMed	3	32	5.0	1e−2	1e−4
Coauthor-CS	4	128	1.0	1e−3	0.0
Amazon-Photo	4	128	1.0	1e−3	1e-4

Table 4
Hyperparameter settings of MHVGCL-MV.

Datasets	# Augmented views	# Embedding dimension	Temperature	Learning rate	Weight decay
Cora	8	128	5.0	1e−2	5e−5
CiteSeer	9	64	5.0	2e−2	1e−4
PubMed	2	32	2.0	2e−2	1e−4
Coauthor-CS	3	128	2.0	1e−3	1e−4
Amazon-Photo	5	128	5.0	2e−2	0

The time complexity of a linear transformation is $O(NFF')$. And the time complexity for generating the kth view is $O\left(N^2F'\right)$. Therefore, the time complexity for generating all the K augmented views is $O\left(NF'(F+NK)\right)$. The time complexity for calculating the multi-hop contrastive loss is $O\left(N^2F'(K-1)\right)$. Then the overall time complexity of MHVGCL is $O\left(NFF' + N^2F'(2K-1)\right) \approx O\left(NFF' + N^2F'K\right)$. Note that in practice, it is always sufficient to take $K \leq 10$, which is relatively small, so the time complexity of MHVGCL is comparable to the representative GCL methods, e.g., GRACE and GCA. Moreover, since our method makes augmentations on the output embedding, the time complexity is smaller than those directly making augmentations on raw input node features, which have time complexity $O\left(N^2FK\right)$ with $F' \ll F$.

4. Experiments

4.1. Datasets

We evaluate the effectiveness of our proposed model through extensive experiments on five benchmark datasets for semi-supervised node classification task, including three widely-used citation networks, i.e., Cora, CiteSeer, PubMed [27], a co-authorship network, i.e., Coauthor-CS [28], and a product co-purchase network, i.e., Amazon-Photo [28]. The statistics of these datasets are listed in Table 2.

4.2. Baselines

To evaluate the effectiveness of MHVGCL, we compare it with 12 advanced methods for semi-supervised node classification tasks, including 2 semi-supervised GNNs, 2 semi-supervised GCL methods, and 8 self-supervised GCL methods:

- **Semi-supervised GNNs**:
  GCN [8] employs an efficient layer-wise propagation rule based on a first-order approximation of spectral convolutions for node classification on graphs.
  GAT [9] uses a multi-head attention mechanism to dynamically generate aggregated weights for neighboring nodes, overcoming limitations of earlier methods like graph convolutions through masked self-attentional layers.
- **Semi-supervised contrastive methods**:
  CG^3 [29] enriches supervision signals with a semi-supervised contrastive loss that captures relationships between data features and graph topology, improving classification with both labeled and unlabeled data.
  CGPN [30] propagates limited labels across the graph using structure information and employs a contrastive objective to extract supervision from extensive unlabeled data.
- **Self-supervised contrastive methods**:
  GMI [24] measures the correlation between input graphs and high-level hidden representations, maximizing mutual information between input and output of a graph neural encoder regarding node features and topology.
  DGI [23] generates graph views by removing edges and masking node features, applying a contrastive loss to maximize agreement of node embeddings within these views.
  GRACE [15] focuses on node-level contrastive learning, creating graph views by removing edges and masking node features, followed by a contrastive loss to maximize node embedding agreement.
  MVGRL [26] learns node and graph-level representations by contrasting encodings from two structure views, including first-order neighbors and graph diffusion.
  GCA [14] maximizes agreement of node embeddings between views generated through adaptive graph augmentation, incorporating diverse priors to address both topological and semantic aspects.
  AFGRL [31] creates an alternative view by identifying nodes that serve as positive samples, considering both local structure and global semantics of graphs.
  SUGRL [32] introduces two triplet losses to explore complementary information between structure and neighbor information, aiming to increase interclass variation and reduce intraclass variation with an upper bound loss.
  NCLA [22] uses a multi-head graph attention mechanism to learn graph augmented views with adaptive topology, compatible with diverse datasets without prior domain knowledge. It employs a neighbor contrastive loss to enable multiple positives per anchor, using network topology as supervised signals.

4.3. Experimental settings

The MHVGCL model proposed in this work was implemented using PyTorch (Version 1.13.0) and Torch-geometric (Version 2.4.0). MHVGCL was trained by the Adam optimizer. In our experiments, we fixed the control factor $\alpha = 0.9$. The number of epochs was fixed as 2000, and the dropout was fixed as 0.5, respectively. To provide fair comparisons, in line with the settings of MVGCL and NCLA, we employed grid search to select the hyperparameters from the following ranges for all datasets without fine-tuning. We chose the learning rate from $\{1e-2, 1e-3\}$ and weight decay from $\{0, 1e-4, 5e-5\}$, respectively. We also used early stopping with a patience of 100. We set the size of the hidden dimension of node embedding from $\{32, 64, 128\}$. The temperature value of the loss function was chosen from $\{0.5, 1, 2, 5\}$. The number of heads was varied from 2 to 10 to investigate its effect. With the above hyperparameter settings, the results have already outperformed the results of the previous methods. The selected hyperparameters applied to MHVGCL-2V and MHVGCL-MV across the five datasets were presented in Table 3 and Table 4, respectively. For the ablation study to investigate the impact of the hyperparameters, we fixed the learning rate at 0.01, the hidden dimension of node embedding at 128, the temperature value at 2, and the number of heads at 3 for all datasets, varying only the hyperparameter under examination across multiple values, while keeping the remaining hyperparameters unchanged. For the self-supervised GCL baselines and

Table 5
Performance on five datasets with different quantities of labeled nodes within each class. Mean accuracy (%) ± standard deviation is shown for each method. The best and second-best results are highlighted in boldface and underlined respectively.

Datasets	c	Methods						
		GCN	GAT	CGPN	CG^3	DGI	GMI	MVGRL
Cora	1	42.6 ± 11.6	42.1 ± 9.5	58.6 ± 10.6	55.4 ± 14.3	55.4 ± 11.4	55.9 ± 9.6	59.1 ± 10.9
	2	55.0 ± 7.5	53.2 ± 9.0	67.4 ± 7.1	66.4 ± 7.7	64.9 ± 9.0	65.2 ± 7.6	67.8 ± 8.6
	3	63.1 ± 6.8	63.2 ± 5.3	70.7 ± 4.0	71.5 ± 4.2	71.1 ± 5.6	70.7 ± 5.2	74.5 ± 4.1
	4	66.4 ± 6.4	66.3 ± 5.9	70.7 ± 2.9	72.7 ± 2.4	72.9 ± 4.5	73.3 ± 4.3	76.1 ± 3.2
	20	79.6 ± 1.8	81.2 ± 1.6	74.0 ± 1.7	80.6 ± 1.6	82.1 ± 1.3	79.4 ± 1.2	**82.4 ± 1.5**
CiteSeer	1	33.8 ± 5.9	31.0 ± 7.2	48.6 ± 11.3	48.4 ± 12.8	47.2 ± 9.2	40.8 ± 6.8	32.8 ± 8.4
	2	44.8 ± 5.5	41.1 ± 7.2	58.0 ± 5.1	60.2 ± 6.8	58.6 ± 4.3	50.2 ± 4.1	47.8 ± 7.5
	3	49.2 ± 5.1	48.6 ± 6.7	59.4 ± 5.4	62.1 ± 7.3	63.3 ± 4.3	55.1 ± 2.7	55.2 ± 6.7
	4	51.7 ± 4.5	52.8 ± 6.6	60.6 ± 3.4	65.1 ± 2.5	65.8 ± 2.1	57.9 ± 3.0	59.3 ± 5.5
	20	66.0 ± 1.2	68.9 ± 1.8	63.7 ± 1.6	70.9 ± 1.5	71.6 ± 1.2	66.9 ± 2.2	71.1 ± 1.4
PubMed	1	48.6 ± 7.1	47.9 ± 8.5	53.5 ± 13.4	54.7 ± 8.6	50.0 ± 9.5	53.5 ± 11.9	55.3 ± 9.3
	2	55.8 ± 7.1	54.5 ± 7.7	59.7 ± 10.3	58.9 ± 7.2	58.5 ± 8.7	60.7 ± 9.9	62.7 ± 7.0
	3	62.1 ± 7.3	61.5 ± 6.8	61.8 ± 10.4	65.1 ± 6.5	62.4 ± 7.2	65.5 ± 8.9	68.5 ± 5.8
	4	65.1 ± 5.9	64.2 ± 6.1	62.7 ± 10.3	66.0 ± 5.7	64.1 ± 6.2	67.2 ± 8.1	70.6 ± 6.0
	20	79.0 ± 2.5	78.5 ± 1.8	73.3 ± 2.5	78.9 ± 2.6	78.3 ± 2.4	76.8 ± 2.3	79.5 ± 2.2
Coauthor-CS	1	64.8 ± 8.8	64.2 ± 9.0	68.4 ± 8.9	79.8 ± 8.0	71.4 ± 6.3	68.3 ± 7.2	75.4 ± 7.2
	2	79.2 ± 4.2	80.2 ± 4.1	77.7 ± 5.3	85.3 ± 4.0	79.6 ± 5.3	78.1 ± 4.5	84.7 ± 2.7
	3	83.3 ± 4.0	85.0 ± 2.7	80.4 ± 4.4	87.5 ± 3.9	82.3 ± 3.6	80.9 ± 4.4	87.5 ± 2.2
	4	84.2 ± 3.1	86.6 ± 2.1	80.9 ± 3.6	87.1 ± 4.6	84.8 ± 2.8	82.8 ± 2.8	88.5 ± 1.8
	20	90.0 ± 0.6	90.9 ± 0.7	83.5 ± 1.4	90.6 ± 1.0	92.0 ± 0.5	88.5 ± 0.8	91.5 ± 0.6
Amazon-Photo	1	60.7 ± 9.3	59.0 ± 11.5	70.4 ± 7.2	69.3 ± 5.8	53.8 ± 10.7	58.2 ± 8.1	59.7 ± 9.0
	2	75.2 ± 7.2	71.7 ± 6.4	75.7 ± 4.3	77.2 ± 3.6	62.7 ± 8.5	68.8 ± 6.2	73.4 ± 6.8
	3	76.9 ± 5.1	75.6 ± 6.3	77.0 ± 4.0	79.4 ± 3.9	66.6 ± 7.7	71.9 ± 5.4	76.8 ± 6.1
	4	81.0 ± 4.6	79.3 ± 5.9	80.1 ± 2.6	81.9 ± 2.9	70.8 ± 6.0	76.2 ± 1.8	82.0 ± 2.3
	20	86.3 ± 1.6	86.5 ± 2.1	84.1 ± 1.5	89.4 ± 1.9	83.5 ± 1.2	86.7 ± 1.5	89.7 ± 1.2

Datasets	c	Methods						
		GRACE	GCA	SUGRL	AFGRL	NCLA	MHVGCL-MV	MHVGCL-2V
Cora	1	51.0 ± 9.8	58.4 ± 10.9	55.2 ± 8.9	47.7 ± 7.8	63.1 ± 11.2	64.9 ± 11.9	**65.5 ± 10.2**
	2	59.7 ± 7.9	66.0 ± 7.8	65.3 ± 6.2	57.8 ± 6.8	71.8 ± 6.9	**72.3 ± 9.1**	71.7 ± 9.8
	3	64.0 ± 6.6	71.5 ± 4.6	70.5 ± 3.5	64.6 ± 4.7	75.7 ± 5.0	**77.1 ± 4.2**	76.5 ± 3.6
	4	66.1 ± 5.4	72.9 ± 4.3	73.5 ± 2.9	67.5 ± 4.2	77.3 ± 3.8	**78.2 ± 3.7**	77.4 ± 3.4
	20	79.6 ± 1.4	79.0 ± 1.4	81.3 ± 1.2	78.6 ± 1.3	82.2 ± 1.6	82.0 ± 1.3	81.4 ± 1.5
CiteSeer	1	40.3 ± 7.2	38.7 ± 9.0	46.7 ± 8.4	42.1 ± 7.2	52.2 ± 13.5	**56.3 ± 12.6**	54.3 ± 13.3
	2	48.5 ± 6.0	49.6 ± 5.3	57.7 ± 4.6	53.3 ± 5.4	62.2 ± 6.4	64.0 ± 6.4	**65.1 ± 6.9**
	3	52.7 ± 4.6	54.2 ± 4.7	61.8 ± 5.1	58.0 ± 4.4	65.5 ± 3.5	66.7 ± 3.3	**67.7 ± 3.7**
	4	56.0 ± 3.9	57.3 ± 3.3	65.0 ± 2.6	61.5 ± 2.5	67.6 ± 2.1	67.9 ± 2.4	**69.8 ± 1.9**
	20	67.0 ± 1.7	65.6 ± 2.4	71.0 ± 1.8	70.8 ± 2.1	71.7 ± 0.9	**73.0 ± 1.3**	72.4 ± 2.0
PubMed	1	46.5 ± 7.0	57.7 ± 10.5	56.7 ± 8.8	49.7 ± 8.3	60.2 ± 12.4	62.5 ± 13.6	**65.4 ± 13.6**
	2	53.8 ± 6.9	66.3 ± 7.6	62.9 ± 6.3	56.4 ± 6.4	66.9 ± 9.7	69.7 ± 8.0	**71.9 ± 10.6**
	3	55.6 ± 7.9	71.9 ± 5.4	67.9 ± 5.7	60.6 ± 5.5	72.3 ± 6.2	74.2 ± 6.7	**75.9 ± 6.2**
	4	57.7 ± 6.8	73.6 ± 5.4	69.9 ± 5.1	62.4 ± 5.1	73.8 ± 4.9	75.9 ± 4.2	**77.4 ± 4.0**
	20	74.6 ± 3.5	81.5 ± 2.5	80.5 ± 1.6	76.4 ± 2.5	82.0 ± 1.4	82.2 ± 1.3	**82.4 ± 1.1**
Coauthor-CS	1	60.0 ± 7.7	59.9 ± 7.6	76.9 ± 6.2	75.2 ± 7.6	83.0 ± 6.2	**84.0 ± 5.1**	83.7 ± 4.9
	2	71.3 ± 4.5	72.5 ± 4.6	85.4 ± 3.1	85.3 ± 2.7	87.4 ± 4.1	**89.1 ± 2.0**	88.8 ± 2.5
	3	74.8 ± 3.8	77.9 ± 4.1	87.4 ± 2.9	87.7 ± 2.3	88.3 ± 3.1	**90.6 ± 1.7**	90.2 ± 2.1
	4	77.6 ± 2.8	80.3 ± 3.1	88.2 ± 2.1	88.4 ± 1.9	88.8 ± 2.4	**90.8 ± 1.4**	90.7 ± 1.6
	20	90.0 ± 0.7	90.9 ± 1.1	91.2 ± 0.9	91.4 ± 0.6	91.5 ± 0.7	92.7 ± 0.5	**92.8 ± 0.5**
Amazon-Photo	1	67.0 ± 9.0	55.3 ± 6.7	71.6 ± 6.2	54.4 ± 9.9	75.6 ± 6.0	**79.7 ± 7.9**	77.2 ± 5.2
	2	76.6 ± 5.2	68.0 ± 5.6	80.7 ± 3.6	71.3 ± 7.2	81.6 ± 3.7	**84.3 ± 4.9**	82.9 ± 3.8
	3	78.6 ± 4.8	74.4 ± 5.9	82.2 ± 2.7	75.9 ± 5.7	83.3 ± 3.8	**85.2 ± 5.7**	85.1 ± 2.5
	4	81.8 ± 1.4	78.8 ± 3.9	84.3 ± 1.6	81.5 ± 2.5	85.3 ± 2.0	**87.5 ± 1.7**	87.4 ± 2.2
	20	87.9 ± 1.4	87.0 ± 1.9	90.5 ± 1.9	89.2 ± 1.1	90.2 ± 1.3	**91.6 ± 1.5**	91.1 ± 1.3

MHVGCL, embeddings were acquired in an unsupervised manner, and subsequently employed to train and test an L_2-regularized logistic regression (LR) classifier for semi-supervised node classification. On one hand, we conducted experiments in the setting of extremely scarce labels, where the number of training nodes per class, denoted by c in Table 5, was selected from the set $\{1, 2, 3, 4\}$. In this case with label scarcity, we followed previous work [33,34] which did not use a validation set with extra labels for model selection. On the other hand, we conducted experiments with relatively sufficient labels. For Cora, CiteSeer, and PubMed, we adopted the method proposed in [35], randomly selecting 20 nodes per class for training, 500 nodes for validation and the remaining nodes for test. For Coauthor-CS and Amazon Photo, we followed previous methodology [36] by randomly selecting 20 nodes per class for training, 30 nodes per class for validation, and the remaining nodes for test. Regarding the self-supervised GCL baselines and MHVGCL, the validation set was only utilized for tuning hyperparameters of the LR classifier, and the C parameter of the LR classifier was from $\{1e-4, 1e-3, 1e-2, 0.1, 1, 10, 100\}$. For each

Table 6
Training time (seconds, mean ± std) over 500 epochs with 10 runs: NCLA, MHVGCL-MV, and MHVGCL-2V on Cora dataset with varying numbers of augmented views.

	2	3	4	5	6	7	8	9	10
NCLA	3.36 ± 0.22	5.21 ± 0.14	7.10 ± 0.04	9.39 ± 0.26	12.93 ± 0.45	16.99 ± 0.90	19.45 ± 0.51	21.15 ± 0.69	23.70 ± 0.34
MHVGCL-MV	1.85 ± 0.21	3.07 ± 0.02	4.06 ± 0.19	5.39 ± 0.03	6.94 ± 0.12	8.19 ± 0.14	10.15 ± 0.07	11.09 ± 0.32	13.65 ± 0.22
MHVGCL-2V	1.50 ± 0.01	1.61 ± 0.01	1.67 ± 0.02	1.76 ± 0.02	1.85 ± 0.01	1.91 ± 0.02	1.98 ± 0.02	2.01 ± 0.01	2.13 ± 0.02

Table 7
Training GPU memory usage (MB) comparison: NCLA, MHVGCL-MV, and MHVGCL-2V on Cora dataset with different number of augmented views.

	2	3	4	5	6	7	8	9	10
NCLA	1666	1866	2066	2240	2440	2612	2812	2984	3186
MHVGCL-MV	1474	1614	1726	1866	1978	2118	2230	2372	2484
MHVGCL-2V	1474	1474	1502	1502	1502	1502	1502	1502	1502

dataset, we conducted 20 random splits of training/validation/test, and reported the averaged performance across all algorithms on the same random splits.

5. Results

5.1. Node classification performance

Table 5 presents the node classification results of our MHVGCL-2V on five benchmark graph datasets, compared to the performance of other methods. The reported results for MHVGCL-2V highlight its ability to achieve a better trade-off between performance and efficiency, even though MHVGCL-MV attains higher performance. These conclusions are drawn from the following sections of this paper. From these results, we can make the following key observations:

First, the results comprehensively validate the superiority of our proposed approach for semi-supervised node classification, which attains superior accuracy across diverse benchmark datasets and varying label scarcity. Specifically, our method consistently outperforms existing techniques on most settings, frequently by significant margins. There are even noticeable gaps (up to 2%) between our approach and leading alternatives (e.g. SUGRL, NCLA) on certain datasets like CiteSeer and Amazon-Photo. This implies prospects for refining methods customized to the specific characteristics and distributions of the dataset, which will be explored in future research. And the overall superior and stable performance of our approach across various benchmarks reinforces its reliability.

Furthermore, under highly limited labeling conditions, our method substantially outperforms existing advanced methods, surpassing competitors by over 5% on PubMed with only 1 labeled node per class. Meanwhile, the node classification accuracy of all methods improves steadily as more labeled data become available, validating the benefits of increased supervised signals for semi-supervised learning. However, as more labels are available, the performance gap gradually decreases. The results strongly demonstrate the MHVGCL's excellent ability to generalize effectively even with limited supervision. This is attributed to its effectiveness in propagating multi-hop information and learning robust representations.

Since the node embedding $\mathbf{H}^{(0)}$ contains no neighborhood information, we tried removing $\mathbf{H}^{(0)}$ in the versions MHVGCL-MV/w0 and MHVGCL-2V/w0, and provide the node class performance of MHVGCL-MV and its version MHVGCL-MV/w0, as well as MHVGCL-2V and its version MHVGCL-2V/w0 in Fig. 2. As the results show, these versions perform almost the same, with MHVGCL-MV(/w0) performing slightly higher than MHVGCL-2V(/w0) in the majority of cases.

In summary, the extensive experiments adequately reveal our method's strengths, especially for generalization given scarce labels. The results provide valuable insights into advancing semi-supervised node classification on graphs.

5.2. Computational and memory efficiency comparison

In this section, we compare self-supervised methods where the training phase and downstream test tasks are separate. To ensure a fair comparison during training, we set the embedding dimension to 128 across all models, with other settings as default per baseline recommendations. For downstream tasks, all methods use the same logistic regression (LR) classifier. Since computational and memory requirements primarily depend on the embedding dimension, our efficiency comparison focuses on the training phase, excluding downstream tasks.

5.2.1. Comparison with NCLA

Among the baselines, only NCLA can perform contrastive learning with an arbitrary number of views. As claimed, NCLA is an efficient graph contrastive learning (GCL) method that does not require additional augmentation operations. Therefore, we first compare our method with NCLA using the same number of contrastive views.

Table 6 presents a comparative analysis of the training time (in seconds, showing mean and standard deviation) over 500 epochs for three methods: NCLA, MHVGCL-MV, and MHVGCL-2V. This comparison is conducted on the Cora dataset, with the number of augmented views ranging from 2 to 10. The results indicate that the MHVGCL-MV method outperforms the NCLA method in terms of training time, with its training duration increasing at a lower rate as more augmented views are introduced. Notably, the MHVGCL-2V method emerges as the most efficient among the three, exhibiting only a modest increase in training time as the number of augmented views increases.

Furthermore, the Table 7 highlights the memory consumption characteristics of these methods during the training process. The MHVGCL-2V method demonstrates superior memory efficiency, maintaining a relatively stable memory footprint across different parameter counts. In contrast, the NCLA method exhibits the highest memory utilization, potentially posing challenges for resource-constrained scenarios or large-scale model training. The MHVGCL-MV method strikes a balance, with its memory consumption falling between the other two methods. Since the training and inference time of downstream tasks is mainly determined by the dimension of the embedding $\mathbf{H}$, and MHVGCL-2V contains only two augmented views, its efficiency is evident.

5.2.2. Comparison with other self-supervised methods

To comprehensively evaluate the computational and memory efficiency of our proposed MHVGCL-2V method, we conducted a comparative analysis with other mentioned self-supervised methods, as presented in Table 6. For our method, we report results using 5 augmented views, which we found to offer an optimal balance between performance and efficiency.

Our method achieves an average training time of 1.76 s (±0.02) over 500 epochs and GPU memory consumption of 1502 MB, outperforming most existing approaches in terms of efficiency. Notably, it shows a 5.4% improvement in training speed compared to the next fastest method, DGI, while maintaining competitive memory usage. However, it is important to note that DGI's efficiency comes with a trade-off in node classification performance. Our method's remarkably low standard deviation in training time indicates high stability, which are crucial factors in practical applications.

Examining other methods reveals various trade-offs. GRACE and GCA, which require additional data augmentation in each epoch, show increased training times but moderate memory usage. AFGRL, while

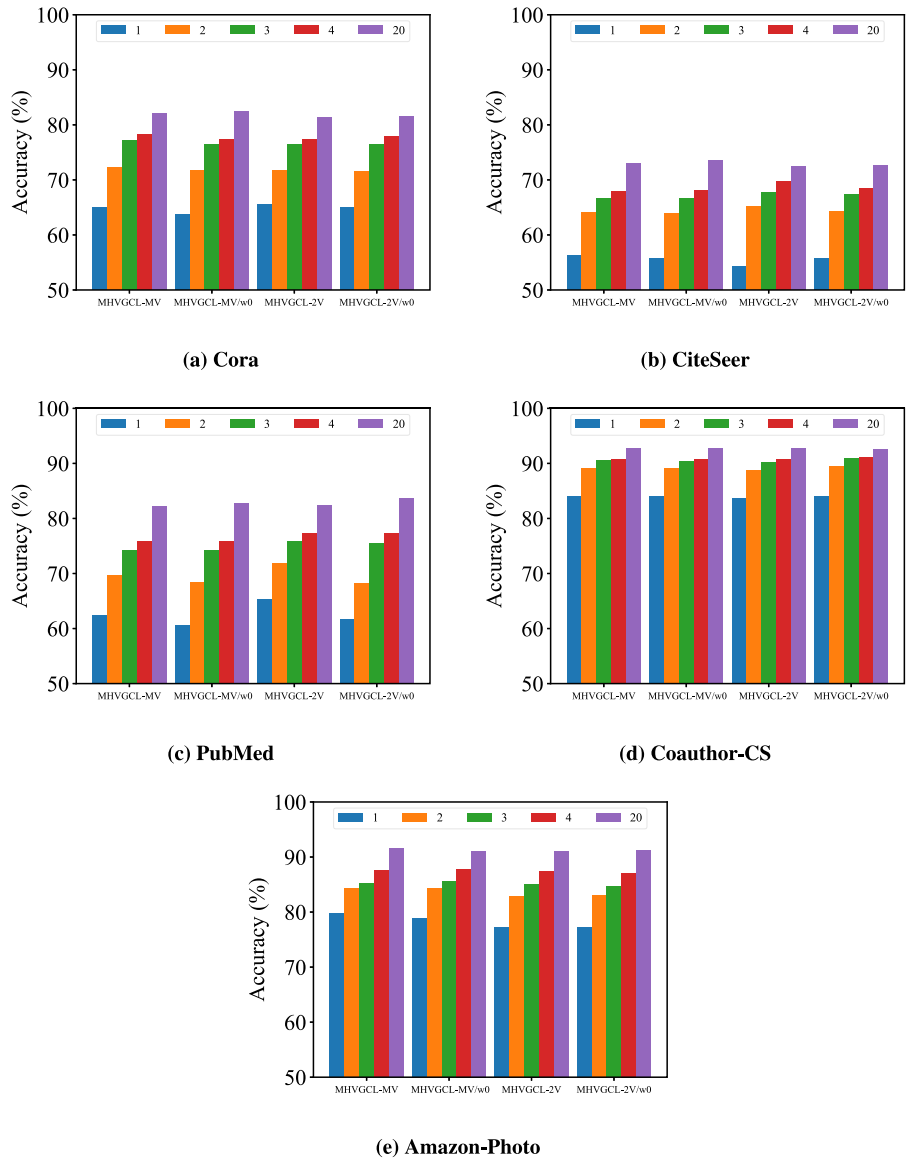


Fig. 2. Performance on five datasets with different quantities of labeled nodes within each class for MHVGCL-MV, MHVGCL-MV/w0, MHVGCL-2V, and MHVGCL-2V/w0, where MHVGCL-MV/w0 and MHVGCL-2V/w0 denote the versions that perform contrastive learning without node embedding $\mathbf{H}^{(0)}$. The legend numbers denote the number of labeled nodes per class in the training set, consistent across Figs. 3, 4, 5, and 6.

efficient in memory usage, has the highest training time due to its k-means-based class center calculations. GMI and MVGRL exhibit higher memory consumption, likely due to their complex architectures involving separate GCN layers and discriminators. It is worth noting that some methods, such as SUGRL, employ an inductive training approach, which fundamentally differs from our transductive method. SUGRL's use of batch data in training results in lower memory overhead, but it struggles with maintaining high performance.

In conclusion, while each method has its strengths and specific use cases, our MHVGCL-2V offers a compelling combination of efficiency, stability, and performance as Table 5 and Table 8 show. However, researchers should carefully select the appropriate number of augmented views to weigh the trade-offs between computational efficiency and task-specific performance metrics for their applications.

5.3. Ablation study

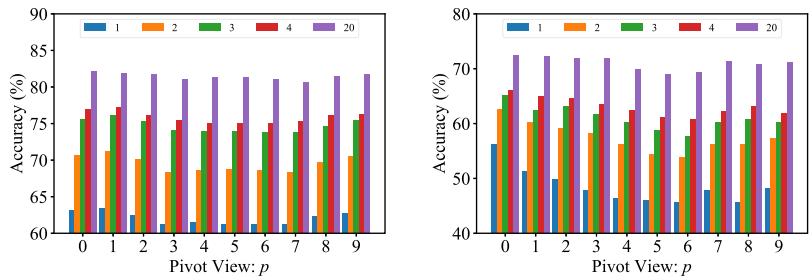
As mentioned earlier, when there are more than two augmented views available, one view can be randomly selected as the pivot view for computing the total multi-hop contrastive loss by summing the contrastive loss between this pivot view and all other views. In this study, we employ the MHVGCL-MV model generated with 9 views on the Cora and CiteSeer dataset. As depicted in Fig. 3, optimal performance is achieved when either the 0-th or 1-th augmentation view is selected as the pivot. This is likely because there is more difference between the kth (with a larger number) view and the 0-th or 1-th view, which contributes more to the contrastive learning process.

Next, we investigate two additional variations of the proposed MHVGCL-2V when utilizing different adjacency matrices. $\mathbf{A}$ and $\hat{\mathbf{A}}$ are the unnormalized adjacency matrix and degree-normalized matrix [8], respectively. As shown in Fig. 4, the utilized standardized normalized adjacency matrix $\tilde{\mathbf{A}}$ consistently yields higher accuracy compared with the unnormalized adjacency matrix variants on the Cora and CiteSeer datasets. While the results for $\hat{\mathbf{A}}$ and $\tilde{\mathbf{A}}$ are comparable, with each having its own advantages.

Additionally, in Fig. 5, the number of augmented views for MHVGCL-2V is increased, but it only concatenates the first and the last view to obtain the final node embedding. The results indicate that

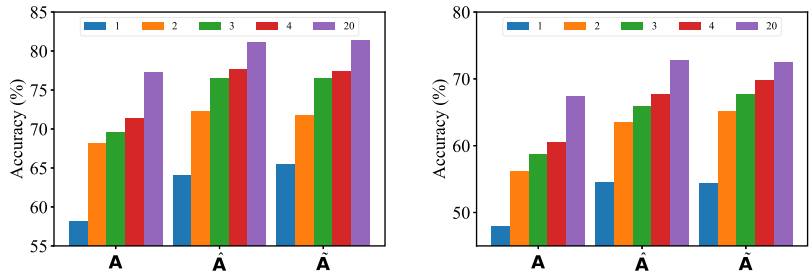
Table 8
Comparison of training time and GPU memory usage on Cora dataset with self-supervised methods. Training time (seconds, mean ± std) over 500 epochs with 10 runs and GPU memory usage (MB).

Datasets	Methods							
	DGI	GMI	MVGRL	GRACE	GCA	SUGRL	AFGRL	MHVGCL-2V
Training time	1.87 ± 0.35	57.81 ± 1.55	4.95 ± 0.31	3.93 ± 0.49	3.95 ± 0.87	2.05 ± 0.44	94.76 ± 0.92	1.76 ± 0.02
Memory usage	1510	1802	2204	1556	1508	1358	1356	1502



(a) MHVGCL-MV (Cora) (b) MHVGCL-MV (CiteSeer)

Fig. 3. Ablation study on the selection of the pivot view..



(a) Cora **(b) CiteSeer**

Fig. 4. Comparisons between variations of the adjacency matrix.

increasing the number of augmented views contributes to improved performance in both MHVGCL-MV and MHVGCL-2V, but it also introduces increased complexity, especially in the MHVGCL-MV version. Therefore, it is recommended to generate 4 or 5 views in MHVGCL for larger datasets to balance performance and complexity.

5.4. Hyperparameter analysis

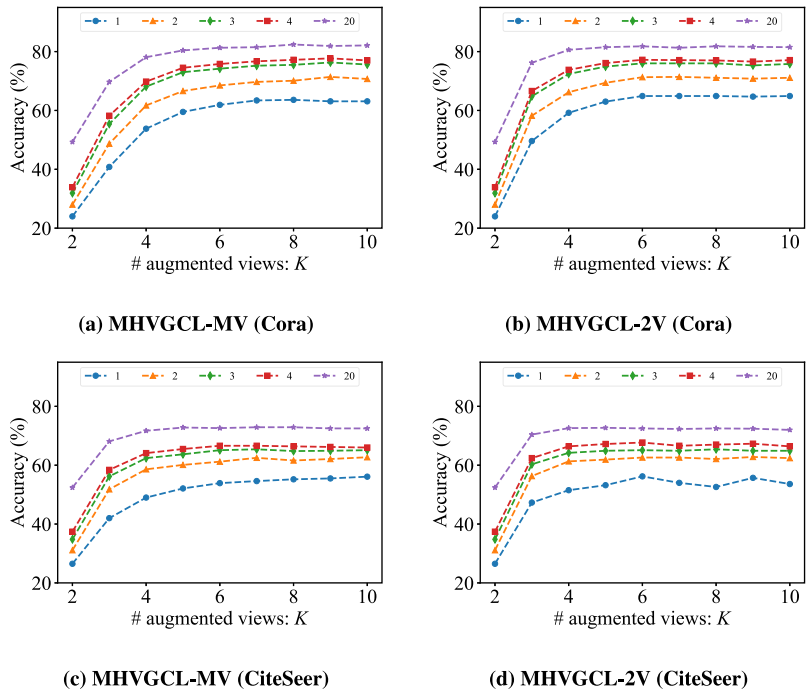
Table 3 and Table 4 show the optimal hyperparameters for MHVGCL-2V and MHVGCL-MV, respectively, due to the differences between the model variants, which result in distinct optimal hyperparameters. Fig. 6 presents the analysis conducted on the hyperparameters F', α, and τ of MHVGCL-2V on the Cora, CiteSeer, and Amazon-Photo datasets, respectively. Moreover, MHVGCL-2V demonstrates stable performance across various embedding dimensions F' within the set $\{16, 32, 64, 128, 256\}$ on the Cora dataset, indicating that the hyperparameter F' has minimal impact on achieving optimal results. The trade-off parameter α, which controls the fusion proportion between consecutive augmented views, reaches its peak performance at around 1.0. When $\alpha = 0$, no hop information is incorporated. Finally, the temperature parameter τ has a significant influence on the results, aligning with the findings of [2,37]. Additionally, setting an appropriately small value for τ enhances the performance of MHVGCL-2V on the Amazon-Photo dataset.

5.5. Visualization of node embeddings

To visually depict and compare the quality of embeddings, Fig. 7 presents t-SNE plots [38] and Calinski–Harabasz index scores [39] for raw data, as well as node embeddings generated by our proposed methods MHVGCL-2V, MHVGCL-MV and the baseline NCLA on the Cora dataset. Notice that points of the same color within a cluster correspond to instances belonging to the same class. The Calinski–Harabasz index, higher index scores indicating better clustering performance, evaluates the quality of clustered node embeddings by considering both the compactness within clusters and the separation between clusters. The comparison of Calinski–Harabasz index scores reveals that MHVGCL-MV achieves higher scores than MHVGCL-2V and NCLA, indicating its superior clustering performance. Although MHVGCL-2V achieves low performance due to the training process involving only two views, we find that concatenating all views in the downstream cluster tasks improves its performance, achieving a Calinski–Harabasz index score of 407.85, which is better than NCLA. The visualization provides a qualitative analysis of the distinguishability of the learned node embeddings, complementing our quantitative evaluations.

5.6. Discussion

In this study, we have demonstrated that MHVGCL achieves strong performance on node classification tasks, particularly on homophilic datasets. When compared with other state-of-the-art graph contrastive learning (GCL) methods, MHVGCL shows competitive advantages due to its utilization of multi-hop views and the novel multi-hop contrastive loss, effectively capturing both local and global topological features. MHVGCL builds upon prior work by introducing multi-hop views generated from a single output head, differentiating it from



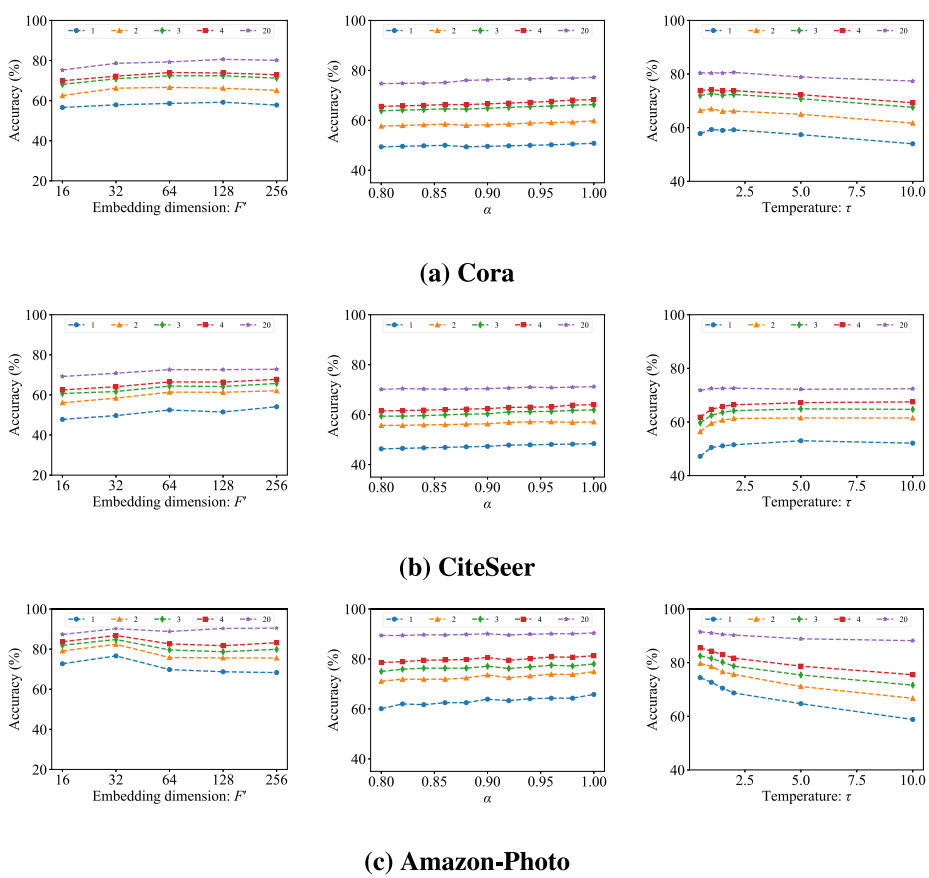
(a) MHVGCL-MV (Cora)

(b) MHVGCL-2V (Cora)

(c) MHVGCL-MV (CiteSeer)

(d) MHVGCL-2V (CiteSeer)

Fig. 5. Analysis on the number of the augmented view.



(a) Cora

(b) CiteSeer

(c) Amazon-Photo

Fig. 6. Analysis of the hyperparameters F', α, and τ in MHVGCL-2V on Cora, CiteSeer and Amazon-Photo datasets.

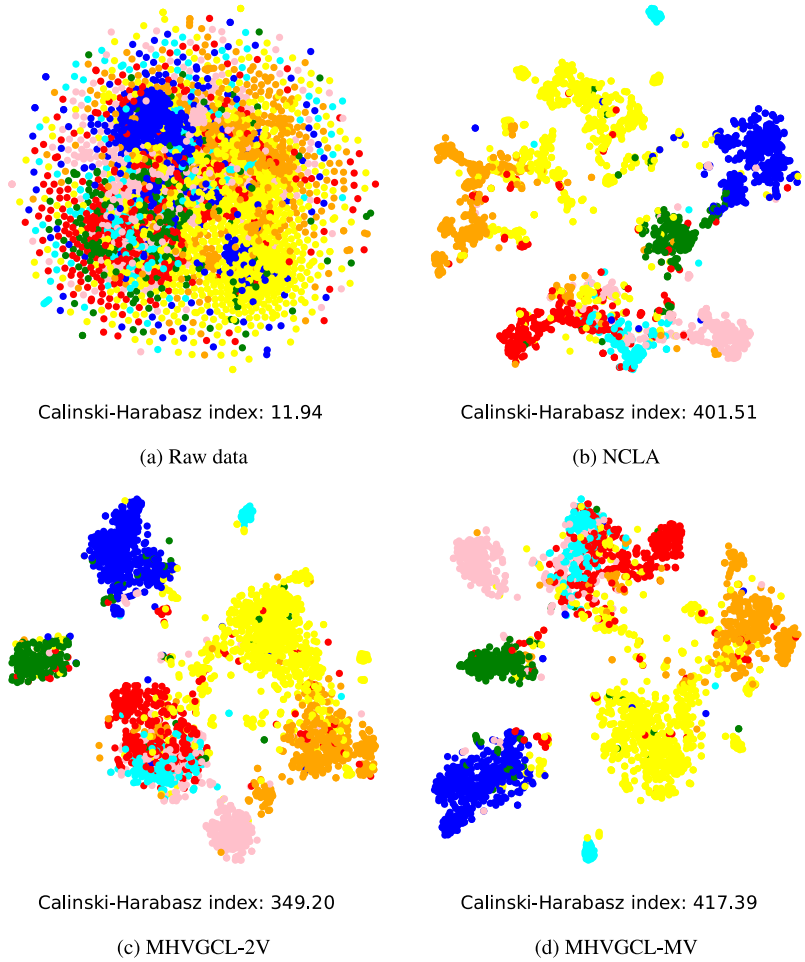


Fig. 7. Visualization of the node embeddings for Cora dataset.

traditional GCL methods that rely on augmenting the raw graph input. In comparison to methods such as GRACE and GCA, which use simpler contrastive mechanisms, MHVGCL's multi-hop neighborhood information offers a more flexible and comprehensive understanding of node relationships, resulting in better discriminative power for node embeddings, especially in cases with scarce labeled data. Furthermore, unlike computationally demanding methods like MVGRL and NCLA, which generate views directly from the input data, MHVGCL reduces memory overhead by generating augmented views after feature transformation. Additionally, a variant of MHVGCL-2V is proposed to further reduce memory and computation time overhead. For instance, datasets like PubMed and Coauthor-CS, which involve large-scale data, benefit significantly from the reduced memory overhead and faster training times of MHVGCL-2V. This results in more efficient learning without sacrificing performance, making MHVGCL well-suited for large-scale graph data, though challenges with extremely large datasets still persist.

However, there are some limitations to our approach. While MHVGCL has shown promising results on homophilic datasets, its performance on heterophilic datasets, where neighboring nodes tend to have different labels, requires further exploration. In such contexts, the multi-hop views may not capture useful signals as effectively, potentially leading to reduced performance. Additionally, MHVGCL's transductive framework poses scalability challenges for extremely large datasets due to memory limitations. Our experiments were conducted on datasets that fit within a 24G GPU, which may limit applicability to graphs of larger scales. This constraint is shared by many transductive GCL methods, and addressing it remains an important direction for future research.

6. Conclusion

GCL is achieved by maximizing mutual information between nodes in augmented graph views generated through graph data augmentation techniques. Despite recent progress, efficiently and effectively generating multiple augmented views to capture comprehensive structural information remains an open challenge in GCL. To address this gap, we propose an innovative graph contrastive learning framework MHVGCL. Our key contributions and results are summarized below:

- MHVGCL produces multi-hop views by exploiting multi-hop neighborhood information, which are obtained iteratively from a single output head. This preserve the graph topology and can bring in both local and global topology and feature information.
- Instead of directly augmenting the raw input data, MHVGCL generates augmented views after feature transformation which reduces the dimensionality of the data and can mitigate the issues of excessive computational cost and memory overhead.
- A novel multi-hop contrastive loss is introduced to learn more robust representations that keeping structural attributes invariant under different augmented views.

The extensive node classification experiments demonstrate that MHVGCL can learn more discriminative node embeddings and surpass current advanced approaches even under highly scarce labeling. Future work will involve extending our experiments to a broader range of heterophilic and more challenging large datasets to validate the generalization and robustness of our method across diverse data landscapes.

CRediT authorship contribution statement

Yutong Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis. **Junheng Zhang:** Writing – review & editing, Validation, Investigation. **Ren Cao:** Writing – review & editing, Validation, Investigation. **Minhao Zou:** Writing – review & editing, Visualization, Validation, Methodology, Investigation, Conceptualization. **Chun Guan:** Writing – review & editing, Validation, Supervision, Methodology. **Siyang Leng:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the STI 2030 - Major Project (No. 2021ZD0201301), the National Natural Science Foundation of China (No. 12471457). This work is also supported by the Science and Technology Commission of Shanghai Municipality (Nos. 2021SHZDZX0103 and 21DZ1201402).

Data availability

Data will be made available on request.

References

[1] Y. Tian, D. Krishnan, P. Isola, Contrastive multiview coding, in: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XI 16, Springer, 2020, pp. 776–794.
[2] K. He, H. Fan, Y. Wu, S. Xie, R. Girshick, Momentum contrast for unsupervised visual representation learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 9729–9738.
[3] L. Wu, Y. Chen, K. Shen, X. Guo, H. Gao, S. Li, J. Pei, B. Long, et al., Graph neural networks for natural language processing: A survey, Found. Trends® Mach. Learn. 16 (2) (2023) 119–328.
[4] T. Shafighfard, F. Kazemi, N. Asgarkhani, D.-Y. Yoo, Machine-learning methods for estimating compressive strength of high-performance alkali-activated concrete, Eng. Appl. Artif. Intell. 136 (2024) 109053.
[5] F. Kazemi, N. Asgarkhani, T. Shafighfard, R. Jankowski, D.-Y. Yoo, Correction: Machine-learning methods for estimating performance of structural concrete members reinforced with fiber-reinforced polymers, Arch. Comput. Methods Eng. (2024) 1.
[6] T. Shafighfard, F. Kazemi, F. Bagherzadeh, M. Mieloszyk, D.-Y. Yoo, Chained machine learning model for predicting load capacity and ductility of steel fiber–reinforced concrete beams, Comput.-Aided Civ. Infrastruct. Eng. (2024).
[7] F. Kazemi, T. Shafighfard, D.-Y. Yoo, Data-driven modeling of mechanical properties of fiber-reinforced concrete: a critical review, Arch. Comput. Methods Eng. 31 (4) (2024) 2049–2078.
[8] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, Int. Conf. Learn. Represent. (ICLR) (2017).
[9] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, Int. Conf. Learn. Represent. (ICLR) (2018).
[10] M. Zou, Z. Gan, R. Cao, C. Guan, S. Leng, Similarity-navigated graph neural networks for node classification, Inform. Sci. 633 (2023) 41–69.
[11] X. Shen, F.-L. Chung, Deep network embedding for graph representation learning in signed networks, IEEE Trans. Cybern. 50 (4) (2018) 1556–1568.
[12] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, Adv. Neural Inf. Process. Syst. 30 (2017).
[13] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, Y. Shen, Graph contrastive learning with augmentations, Adv. Neural Inf. Process. Syst. 33 (2020) 5812–5823.
[14] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Graph contrastive learning with adaptive augmentation, in: Proceedings of the Web Conference 2021, 2021, pp. 2069–2080.
[15] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Deep graph contrastive representation learning, In ICML Work. Graph Represent. Learn. Beyond (2020).
[16] H. Xue, X.-K. Sun, W.-X. Sun, Multi-hop hierarchical graph neural networks, in: 2020 IEEE International Conference on Big Data and Smart Computing (BigComp), IEEE, 2020, pp. 82–89.
[17] G. Wang, R. Ying, J. Huang, J. Leskovec, Multi-hop attention graph neural network, Int. Jt. Conf. Artif. Intell. (IJCAI) (2021) 3089–3096.
[18] C. Sun, J. Hu, H. Gu, J. Chen, M. Yang, Adaptive graph diffusion networks, 2020, arXiv preprint arXiv:2012.15024.
[19] M. Zou, Z. Gan, Y. Wang, J. Zhang, D. Sui, C. Guan, S. Leng, Unig-encoder: A universal feature encoder for graph and hypergraph node classification, Pattern Recognit. 147 (2024) 110115.
[20] X. Chen, K. He, Exploring simple siamese representation learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 15750–15758.
[21] L. Wu, H. Lin, C. Tan, Z. Gao, S.Z. Li, Self-supervised learning on graphs: Contrastive, generative, or predictive, IEEE Trans. Knowl. Data Eng. 35 (4) (2021) 4216–4235.
[22] X. Shen, D. Sun, S. Pan, X. Zhou, L.T. Yang, Neighbor contrastive learning on learnable graph augmentation, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, 2023, pp. 9782–9791.
[23] P. Veličković, W. Fedus, W.L. Hamilton, P. Liò, Y. Bengio, R.D. Hjelm, Deep graph infomax, Int. Conf. Learn. Represent. (ICLR) (2019).
[24] Z. Peng, W. Huang, M. Luo, Q. Zheng, Y. Rong, T. Xu, J. Huang, Graph representation learning via graphical mutual information maximization, in: Proceedings of the Web Conference 2020, 2020, pp. 259–270.
[25] D. Lee, K. Shin, I'm me, we're us, and i'm us: Tri-directional contrastive learning on hypergraphs, in: Proceedings of the AAAI Conference on Artificial Intelligence, 37, 2023, pp. 8456–8464.
[26] K. Hassani, A.H. Khasahmadi, Contrastive multi-view representation learning on graphs, in: International Conference on Machine Learning, PMLR, 2020, pp. 4116–4126.
[27] H. Pei, B. Wei, K.C.-C. Chang, Y. Lei, B. Yang, Geom-GCN: Geometric graph convolutional networks, Int. Conf. Learn. Represent. (ICLR) (2020).
[28] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, 2018, arXiv preprint arXiv:1811.05868.
[29] S. Wan, S. Pan, J. Yang, C. Gong, Contrastive and generative graph convolutional networks for graph-based semi-supervised learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, 2021, pp. 10049–10057.
[30] S. Wan, Y. Zhan, L. Liu, B. Yu, S. Pan, C. Gong, Contrastive graph poisson networks: Semi-supervised learning with extremely limited labels, Adv. Neural Inf. Process. Syst. 34 (2021) 6316–6327.
[31] N. Lee, J. Lee, C. Park, Augmentation-free self-supervised learning on graphs, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, 2022, pp. 7372–7380.
[32] Y. Mo, L. Peng, J. Xu, X. Shi, X. Zhu, Simple unsupervised graph representation learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, 2022, pp. 7797–7805.
[33] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, 2018.
[34] Q. Li, X.-M. Wu, H. Liu, X. Zhang, Z. Guan, Label efficient semi-supervised learning via graph filtering, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 9582–9591.
[35] Z. Yang, W. Cohen, R. Salakhudinov, Revisiting semi-supervised learning with graph embeddings, in: International Conference on Machine Learning, PMLR, 2016, pp. 40–48.
[36] M. Liu, H. Gao, S. Ji, Towards deeper graph neural networks, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 338–348.
[37] A.v.d. Oord, Y. Li, O. Vinyals, Representation learning with contrastive predictive coding, 2018, arXiv preprint arXiv:1807.03748.
[38] L. Van der Maaten, G. Hinton, Visualizing data using t-sne., J. Mach. Learn. Res. 9 (11) (2008).
[39] T. Caliński, J. Harabasz, A dendrite method for cluster analysis, Comm. Statist. Theory Methods 3 (1) (1974) 1–27.